Compound Component Pattern

2 min read Updated 24 Mar 2026 Topic 33 of 62

Bookmark Mark done

6.4 – Compound Component Pattern

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is the Compound Component pattern?"	🔴	★★★☆☆
"How do compound components share state?"	🔴	★★★☆☆
"Give an example of compound components"	🔴	★★★☆☆

The Pattern

Compound components are a set of components that work together to accomplish a single task, sharing implicit state between them. The parent component manages the state, and children access it through Context, without the consumer needing to wire up props manually.

Think of `<select>` and `<option>` in HTML. They're separate elements but work together implicitly. The `<select>` manages which option is selected, and each `<option>` participates without explicit prop passing.

Building a Compound Tab Component

JSX Copy

```jsx
const TabsContext = createContext();

function Tabs({ children, defaultIndex = 0 }) {
  const [activeIndex, setActiveIndex] = useState(defaultIndex);

  return (
    <TabsContext.Provider value={{ activeIndex, setActiveIndex }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

function TabList({ children }) {
  return <div className="tab-list" role="tablist">{children}</div>;
}

function Tab({ children, index }) {
  const { activeIndex, setActiveIndex } = useContext(TabsContext);

  return (
    <button
      role="tab"
      className={activeIndex === index ? 'tab active' : 'tab'}
      onClick={() => setActiveIndex(index)}
    >
      {children}
    </button>
  );
}

function TabPanels({ children }) {
  const { activeIndex } = useContext(TabsContext);
  return <div className="tab-panels">{children[activeIndex]}</div>;
}
```

```
}
```

The beauty is in the usage

JSX Copy

```jsx
<Tabs defaultIndex={0}>
  <TabList>
    <Tab index={0}>Profile</Tab>
    <Tab index={1}>Settings</Tab>
    <Tab index={2}>Billing</Tab>
  </TabList>

  <TabPanels>
    <TabPanel><ProfilePage /></TabPanel>
    <TabPanel><SettingsPage /></TabPanel>
    <TabPanel><BillingPage /></TabPanel>
  </TabPanels>
</Tabs>
```

The consumer doesn't pass `activeIndex` or `setActiveIndex` anywhere. The components communicate implicitly through Context. This is what makes compound components feel natural and declarative.

Where You See This Pattern

Libraries that use compound components include: Headless UI (`<Menu>`, `<Listbox>`), Reach UI, Radix UI, and Chakra UI's `<Accordion>`, `<Tabs>`, `<Menu>`.

Interview answer: "Compound components are a group of components that share implicit state through Context. The parent manages the state, and children consume it without the user needing to pass props explicitly. It's like how HTML's `<select>` and `<option>` work together. I've used this pattern to build tabs, accordions, and dropdown menus where the API should be declarative and composable. Libraries like Headless UI and Radix use this pattern extensively."

Part of the React Interview Preparation Kit — Back to Course Roadmap →

QUICK CHECK — no marks, just for you.

1. According to the lesson, how do compound components share state between the parent and its children?

- ○ The parent holds the state and children read it through Context, so the consumer never wires up props manually
- ○ The state lives in a module-level variable that every child imports directly
- ○ Each child holds its own copy of the state and the parent syncs them with useEffect
- ○ The parent clones every child with React.cloneElement and injects the state as explicit props

Check answer

2. The lesson uses HTML's <select> and <option> as an analogy for compound components. What specifically makes them a good analogy?

- ○ <option> passes the selected value up to <select> through a callback attribute
- ○ <select> and <option> can each be used independently and still render correctly on their own
- ○ <select> manages which option is selected, and each <option> participates without explicit prop passing
- ○ Both elements are rendered by the browser without any shared state at all

Check answer

3. [illegible] example, which statement about what the consumer writes is correct?

- The consumer must pass activeIndex down to every Tab so it knows whether it is selected
- The consumer must wrap the markup in a TabsContext.Provider themselves before using Tabs
- The consumer must pass setActiveIndex to TabList so clicks can bubble up to Tabs
- The consumer never passes activeIndex or setActiveIndex anywhere; the pieces communicate through Context

Check answer

‹ Previous
Render Props Pattern

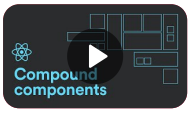
Next ›
Provider Pattern & Composition Pattern

LEARN MORE

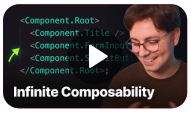
ARTICLES & DOCS

Compound Pattern (patterns.dev)
patterns.dev

VIDEOS



Compound components pattern
YouTube



The magic of compound components
YouTube





Compound components (design patterns)

React.js Interview Kit

Hand-picked references for this lesson.